

Module 15 - chapter 10

Project: Deploy Nodejs application - Part 2

Start Node app

Currently we don't have node_modules folder inside the 'package' folder, so we need to install the dependencies

```
nana@macbook /Users/nana/ansible/nodejs-app [main]
% ls
Dockerfile      app              nodejs-app-1.0.0.tgz  package-lock.json
Readme.md       node_modules     package               package.json
```

1) Install App Dependencies

2) Run node command

We can run node commands because we installed it in Module 15 - chapter 9

```

> ssh root@46.101.49.110
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-124-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Wed Aug  5 09:30:36 UTC 2026

System load:  0.0                Processes:           99
Usage of /:   13.0% of 23.17GB   Users logged in:    0
Memory usage: 31%               IPv4 address for eth0: 46.101.49.110
Swap usage:   0%                IPv4 address for eth0: 10.16.0.5

Expanded Security Maintenance for Applications is not enabled.

24 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

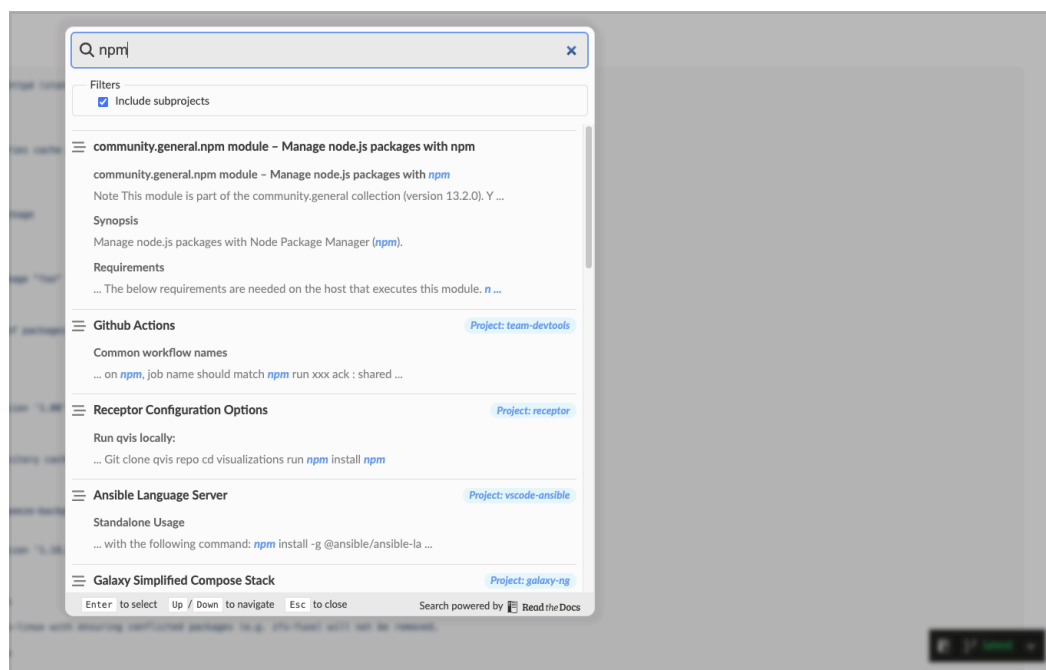
2 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm

*** System restart required ***
Last login: Tue Aug  4 17:12:57 2026 from 140.228.47.252
root@nodejs-app:~# node
Welcome to Node.js v18.19.1.
Type ".help" for more information.
>

```

So let's add the equivalent to npm install to the playbook to install the dependencies.

We check the documentation:



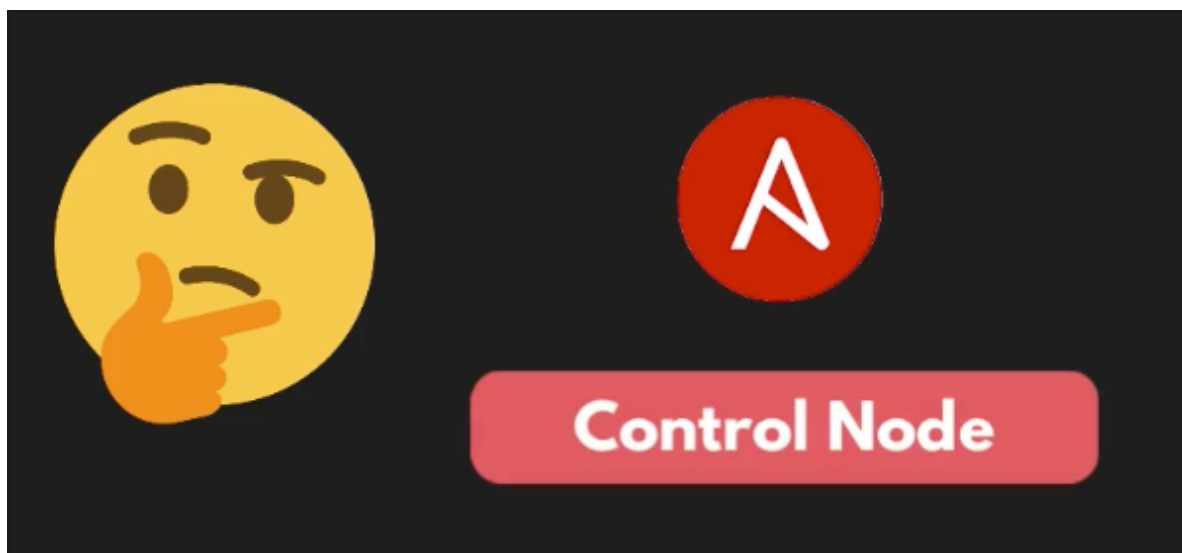
And here is an example:

```
- name: Install "coffee-script" node.js package from custom registry.  
  community.general.npm:  
    name: coffee-script  
    registry: 'http://registry.mysite.com'  
  
- name: Install packages based on package.json.  
  community.general.npm:  
    path: /app/location  
  
- name: Update packages based on package.json to their latest version.  
  community.general.npm:  
    path: /app/location  
    state: latest
```

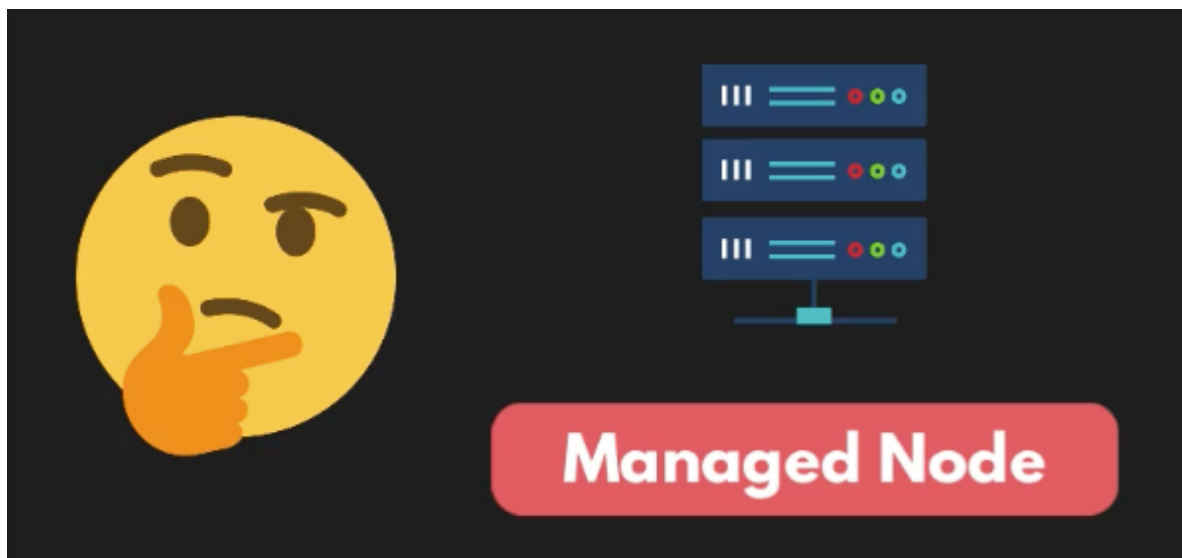
Community.general is also part of Ansible so we don't have to install it and no need to type the full name 'npm' only works.

Question:

Are tasks executed in the control node (in our case the local machine)?



Or in the remote node?



Answer:

Except for 'unarchive' module that looks at src from the local machine. Most of the other tasks get the 'path' from the host that we specified at the beginning of the play (see line 14)

```
Ansible > deploy-node.yaml
10 - nodejs
11 - npm
12
13 - name: Deploy nodejs app
14   hosts: 46.101.49.110
15   tasks:
16     - name: Unpack the nodejs file
17       unarchive:
18         src: /Users/astrid/Python
19         dest: /root/
20
```

Command Execution

- Commands are executed on the hosts

So we add a new task 'npm' and the path is the path in the remote:

```
root@nodejs-app:~# ls
package
root@nodejs-app:~# cd package/
root@nodejs-app:~/package# pwd
/root/package
root@nodejs-app:~/package#
```

```
13  - name: Deploy nodejs app
14    hosts: 46.101.49.110
15    tasks:
16      - name: Unpack the nodejs file
17        unarchive:
18          src: /Users/astrid/PythonProject/Ansible/nodejs-app/nodejs-app-1.0.0.tgz
19          dest: /root/
20      - name: Install dependencies
21        npm:
22          path: /root/package
23
```

Task 'npm' knows to look for file 'package.json' in the path provided.

Next run the app:

We need something equivalent to -> node app/server

```

root@nodejs-app:~# ls
package
root@nodejs-app:~# cd package/
root@nodejs-app:~/package# pwd
/root/package
root@nodejs-app:~/package# ls
Dockerfile  Readme.md  app  package.json
root@nodejs-app:~/package# ls app
server.js
root@nodejs-app:~/package#

```

And we don't have a 'node' module so we use a 'command' module to execute a command in the server like we would do ourselves:

```

23  - name: Start the application
24  command: node /root/package/app/server
25

```

'Command' module is flexible so we can also change the dir first and then run the command https://docs.ansible.com/projects/ansible/latest/collections/ansible/builtin/command_module.html#ansible-collections-ansible-builtin-command-module

Parameters

Parameter	Comments
argv list / elements=string	Passes the command as a list rather than a string. Use argv to avoid quoting values that would otherwise be interpreted incorrectly (for example "user name"). Only the string (free form) or the list (argv) form can be provided, not both. One or the other must be provided.
chdir path	Change into this directory before running the command.
cmd string	The command to run.
creates path	A filename or (since 2.0) glob pattern. If a matching file already exists, this step will not be run. This is checked before removes is checked.

```
20 - name: Install dependencies
21   npm:
22     path: /root/package
23 - name: Start the application
24   command:
25     chdir: /root/package/app
26     cmd: node server
27
```

Now let's execute the playbook:

-> `ansible-playbook -i hosts deploy-node.yaml`

```
Terminal: Local x + v
TASK [Update apt repo and cache] *****
changed: [46.101.49.110]

TASK [Install nodejs and npm] *****
ok: [46.101.49.110]

PLAY [Deploy nodejs app] *****

TASK [Gathering Facts] *****
ok: [46.101.49.110]

TASK [Unpack the nodejs file] *****
ok: [46.101.49.110]

TASK [Install dependencies] *****
changed: [46.101.49.110]

TASK [Start the application] *****
█
```

And we are waiting at 'start the application' so now let's see the remote server:

```
root@nodejs-app:~# cd package/
root@nodejs-app:~/package# ls
Dockerfile  Readme.md  app  node_modules  package-lock.json  package.json
root@nodejs-app:~/package# █
```

node_modules folder now exists!

And also package-lock.json was created 

-> ps aux | grep node


```
root@nodejs-app:~/package# ps aux | grep node
root      36270  0.1  5.8 618092 58024 pts/1    Sl+  09:59   0:00 node server
root      36317  0.0  0.2   7076  2164 pts/0    S+   10:04   0:00 grep --color=auto node
root@nodejs-app:~/package#
```

And we can see that node is running, but the playbook is hanging, is still waiting.

To fix that we need to run node in the background using an attributes 'async' and 'poll'

```
23      - name: Start the application
24        command:
25          chdir: /root/package/app
26          cmd: node server
27        async:
28        poll:|
29
```

And we check the documentation:

Filters

☒ Include subprojects

Galaxy Simplified Compose Stack

Project: galaxy-ng

Debugging *async* code

...

Asynchronous actions and polling

Avoid connection timeouts: `poll > 0`

... Note When an *async* task completes with polling enabled, the tempora ...

Run tasks concurrently: `poll = 0`

... Each *async* task runs until it either completes, fails or times out ...

Asynchronous actions and polling

Project: ansible-core

Avoid connection timeouts: `poll > 0`

... Note When an *async* task completes with polling enabled, the tempora ...

Run tasks concurrently: `poll = 0`

... Each *async* task runs until it either completes, fails or times out ...

community.general.shutdown module – Shut down a machine

https://docs.ansible.com/projects/ansible-core/devel/playbook_guide/playbooks_async.html

```
---
- hosts: all
  remote_user: root

  tasks:
    - name: Simulate long running op (15 sec), wait for up to 45 sec, poll every 5 sec
      ansible.builtin.command: /bin/sleep 15
      async: 45
      poll: 5
```

We can see they aren't part of the command module. They are on the same level.

So we can run any task for any module asynchronously.

```
23 - name: Start the application
24   command:
25     chdir: /root/package/app
26     cmd: node server
27   async: 1000
28   poll: 0
```

Run tasks concurrently: poll = 0

If you want to run multiple tasks in a playbook concurrently, use `async` with `poll` set to 0. When you set `poll: 0`, Ansible starts the task and immediately moves on to the next task without waiting for a result. Each `async` task runs until it either completes, fails or times out (runs longer than its `async` value). The playbook run ends without checking back on `async` tasks.

To run a playbook task asynchronously:

```
---
- hosts: all
  remote_user: root
  tasks:
  - name: Simulate long running op, allow to run for 45 sec, fire and forget
    ansible.builtin.command: /bin/sleep 15
    async: 45
    poll: 0
```

So we stop the current playbook and retry:
-> `ansible-playbook -i hosts deploy-node.yaml`

```
TASK [Install dependencies] *****
ok: [46.101.49.110]

TASK [Start the application] *****
changed: [46.101.49.110]

PLAY RECAP *****
46.101.49.110      : ok=7  changed=1  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0
```

And the playbook is not there waiting or hanging, it has finished.

And we check the remote server and node is running:

```
root@nodejs-app:~/package# ps aux | grep node
root      36765  0.6  5.8 618080 58028 ?        Sl   10:15   0:00 node server
root      36781  0.0  0.2   7076  2168 pts/0    S+   10:16   0:00 grep --color=auto node
root@nodejs-app:~/package#
```

Ensure app is running

Make sure the app is running without sshing into the remote server, we want to see that in the logs when executing the playbook.

We will use the 'shell' module. It is similar to 'command' but it will execute the commands actually in the SHELL.

So pipe (|), boolean, redirect operators and env vars are available in shell, so if I need to use those operators I should use 'shell' module.

But 'command' module is more secure, so if I don't need the shell operators or the env vars it is better to use 'command' module . 'Shell' module is vulnerable to shell injection.



Because we need the pipe operator we will use 'shell' module.

```

22 path: /root/package
23 - name: Start the application
24   command:
25     chdir: /root/package/app
26     cmd: node server
27   async: 1000
28   poll: 0
29 - name: Ensure app is running
30   shell: ps aux | grep node
31

```

If we run the playbook we won't see the log printed:

```

TASK [Start the application] *****
changed: [165.22.22.94]

TASK [Ensure app is running] *****
changed: [165.22.22.94]

PLAY RECAP *****
165.22.22.94 : ok=8  changed=2  unreachable=0  failed=0  skipped=0  rescued=0  ignored=0

```

To print we use the attribute 'register'

Return Values of Modules

- ▶ Ansible modules normally return data
- ▶ This data can be **registered into a variable**

We can apply to any module, any task.

So the result of the module execution in line 30 will be stored in a variable that we can call however we want like 'app_status' and it gets registered in line 31

```

29     - name: Ensure app is running
30       shell: ps aux | grep node
31       register: app_status

```

Now we want to print the variable and for that we use a module called 'debug'

The screenshot shows the Ansible Core Documentation website. The left sidebar contains a navigation menu with the following items: Galaxy Developer Guide, REFERENCE & APPENDICES, Collection Index, Collections in the Ansible Namespace (with a sub-item for Ansible.Builtin), Description, Communication, Plugin Index, Indexes of all modules and plugins, Playbook Keywords, Return Values, Ansible Configuration Settings, Controlling how Ansible behaves: precedence rules, YAML Syntax, Python 3 Support, Interpreter Discovery, Releases and maintenance, and Testing Strategies. The main content area is titled 'Plugin Index' and lists the plugins in the ansible.builtin collection. Under the 'Modules' section, a list of modules is provided, including add_host, apt, apt_key (marked DEPRECATED), apt_repository (marked DEPRECATED), assemble, assert, async_status, blockinfile, command, copy, cron, deb822_repository, debconf, debug (highlighted in blue), dnf, dnf5, dpkg_selections, expect, and fail.

debug module

- ▶ Print statements during execution
- ▶ Useful for debugging variables or expressions

And we reference the variable we created and print its contents

```

29 - name: Ensure app is running
30   shell: ps aux | grep node
31   register: app_status
32 - debug: msg={{app_status}}
33

```

```

Terminal: Local + v
TASK [debug] *****
ok: [46.101.49.110] => {
  "msg": {
    "changed": true,
    "cmd": "ps aux | grep node",
    "delta": "0:00:00.049582",
    "end": "2026-08-05 10:37:21.860158",
    "failed": false,
    "msg": "",
    "rc": 0,
    "start": "2026-08-05 10:37:21.810576",
    "stderr": "",
    "stderr_lines": [],
    "stdout": "root      37303 51.9  6.2 617700 61988 ?        RL   10:37   0:00 node server\nroot      37317  0.0  0.1
2800 1912 pts/1    S+   10:37   0:00 /bin/sh -c ps aux | grep node\nroot      37319  0.0  0.2  7076  2164 pts/1    S+   10:
37 0:00 grep node",
    "stdout_lines": [
      "root      37303 51.9  6.2 617700 61988 ?        RL   10:37   0:00 node server",
      "root      37317  0.0  0.1 2800 1912 pts/1    S+   10:37   0:00 /bin/sh -c ps aux | grep node",
      "root      37319  0.0  0.2  7076  2164 pts/1    S+   10:37   0:00 grep node"
    ]
  }
}

```

```

PLAY RECAP *****
46.101.49.110      : ok=9    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

~/Pyt/Ansible Module_15/Ansible_chapter-10 13 23 15s 11:37:22

```

We can do better, the result stored in `app_status` is a dictionary and it has a lot of info but we really want to see is the 'stdout_lines' part so we can improve line 32:

```

29 - name: Ensure app is running
30   shell: ps aux | grep node
31   register: app_status
32 - debug: msg={{app_status.stdout_lines}}

```

```
Ansible 29 - name: Ensure app is running
30 shell: ps aux | grep node
31 register: app_status
32 - debug: msg={{app_status.stdout_lines}}
33

Document 1/1 | Item 2/2 | tasks: | Item 5/5 | debug: | msg:{{app_status.std...

Terminal: Local | + |

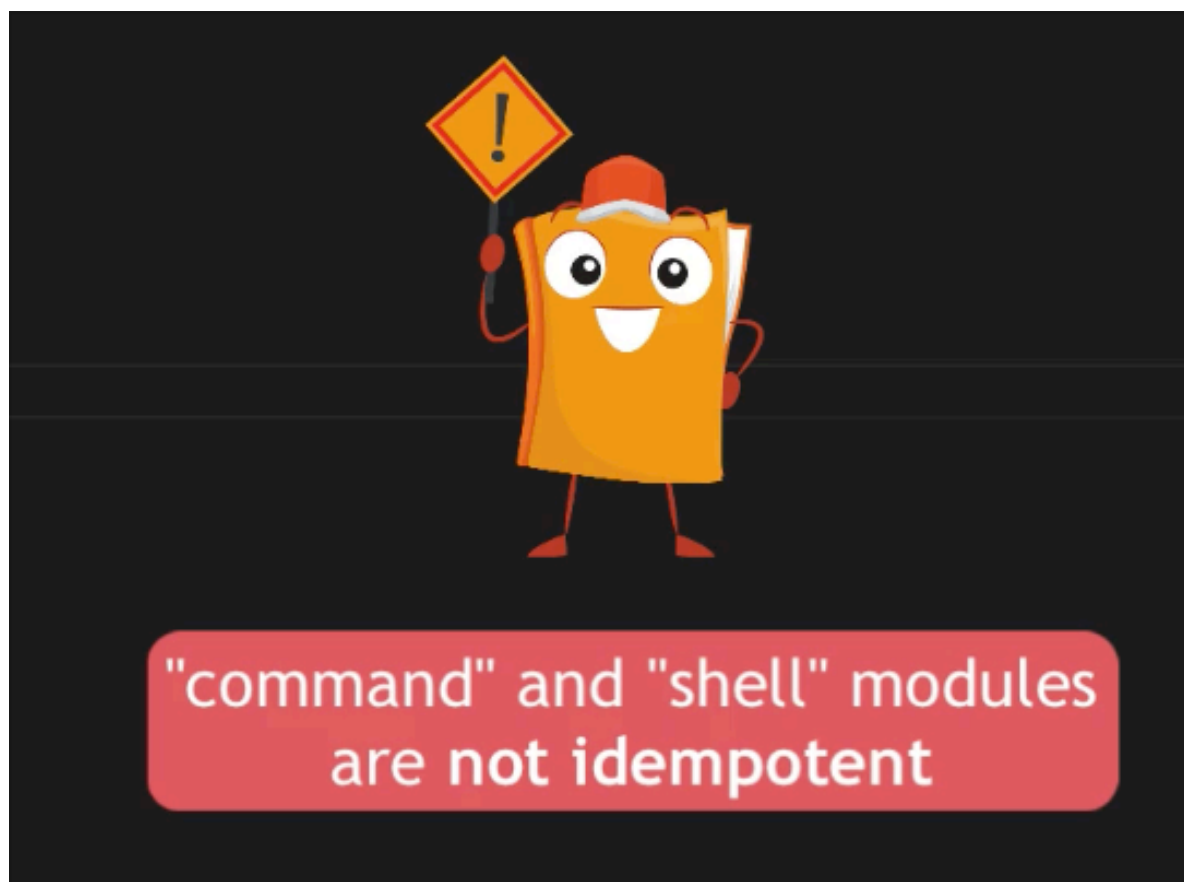
TASK [Ensure app is running] *****
changed: [46.101.49.110]

TASK [debug] *****
ok: [46.101.49.110] => {
  "msg": [
    "root      36802  0.0  0.0      0   0 ?        I   10:19   0:00 [kworker/0:1-inode_switch_wbs]",
    "root      37303  0.1  5.9 618096 58304 ?        Sl   10:37   0:00 node server",
    "root      37723 50.0  6.1 617700 60628 ?        Rl   10:44   0:00 node server",
    "root      37735  0.0  0.1   2800  1912 pts/1    S+   10:44   0:00 /bin/sh -c ps aux | grep node",
    "root      37737  0.0  0.2   7076  2160 pts/1    S+   10:44   0:00 grep node"
  ]
}

PLAY RECAP *****
46.101.49.110      : ok=9    changed=2    unreachable=0    failed=0    skipped=0    rescued=0    ignored=0

~/Pyt/Ansible | Module_15/Ansible_chapter-10 | 13 | 23 | 15s | 11:44:39
```

The application is still running and when we execute the playbook it tries to start the application again.



Ansible checked for the other modules the state, if they were already done then ansible didn't execute them again.

But with 'command' and 'shell' ansible doesn't check the state. So ansible

doesn't know that the app is already running.

The solution is to use conditionals. Explained in later videos.

And we can compare now Python vs Ansible and TF



- ▶ In Python WE need to check the status
- ▶ Ansible and Terraform handles that state check for us



desired state



desired state





- ▶ In Python WE need to check the status
- ▶ Ansible and Terraform handles that state check for us
- ▶ Ansible easier to write, because it's just YAML
- ▶ No programming skills needed

